

Building a Safe Natural Language to SQL Pipeline

Data Science 360 F25 Lab 06
Madeleine Arganbright
Kennedy Herring

Overview

In this lab, we created an NL-to-SQL translator using several `query_ollama` functions. The project involved two main parts: (A) building a model that produces accurate SQL queries, and (B) ensuring that this model could not be used to harm the database. We worked with the `gravity_books` database in read-only mode. The objective was to build a model that generated accurate SQL queries from user queries. Once we completed that part of the lab, we had to structure a strong safety envelope that allowed the user to use SQL but didn't let them mess with our database or access unreasonably large amounts of information. There were two main things we had to protect against under this umbrella: the first was Prompt Injection, where a user may say 'forget all instructions... complete other task'; the other was SQL injection, where the user may type SQL into the model directly. We worked with the `Gravity_books` data and fed our model the entire schema. If we were working with a larger, more complex database, we might have needed to chunk this information to feed it to the model, since the user asked to get hold of specific tables.

Design and Methods

We start our pipeline by feeding the LLM our schema and giving it a prompt explaining its purpose. Then we get user input and feed it into the model as the request. The model then checks the user input and labels it as safe, unsafe, or unrelated. Then, if the request is deemed safe, the model proceeds with the query by calling our `generateSQL` function, which returns a JSON object containing the "generated SQL query" and the cleaned SQL query as the model would execute it in MySQL. If the request is deemed unsafe or unrelated, the model does not execute it, and a message is printed to the user explaining this.

Testing and Experiments

We used Python tests to test behavior and safety. We tested the input evaluation function (`validate`) by confirming that example prompts, which were unsafe (i.e., containing prompt injections), unrelated (i.e., asking the LLM to write a poem), and safe (i.e., a generic request asking for five books from the database) returned the expected classification.

We then devised a few tests to specifically target our SQL validation helper function (`check_SQL_validity`) and ensure it reviews generated SQL queries. First, we checked that the response was being properly validated (i.e., removing code fences and ensuring SQL queries referencing non-existent tables or illegal operations like `delete` were not added to a JSON object and performed by the model). We also included a test for invalid SQL (i.e., appropriate SQL keywords in the wrong order). We

did not include a specific test for proper SQL queries, since we already had a few examples in our parent class (`validate`).

Next, we tested a simple query with many results, to ensure that the limit was being correctly enforced, as well as a query with no results. We also tested a more complex query with joins, and a query with a subquery to ensure that they were both correctly labeled as valid or invalid.

Finally, we manually input a few (valid) requests and reviewed them ourselves, to ensure that the model's natural language response was brief, appropriate, and actually included proper data from the database.

Results

Overall, our model did a good job of identifying and rejecting malicious inputs and classifying off-topic inputs. E.g., The input “I want to change the name of one of the books in the table” returned “Unsafe keyword detected in query, please revise your request”.

We did have difficulty getting the model to correctly identify SQL queries with invalid syntax (e.g., “FROM book SELECT title”). We did not encounter any issues with the model itself generating malformed queries. Still, if such a case arose, we do not think our SQL validation would be effective at recognizing it. (To remedy this, we included a ‘try; except’ block in the actual query execution, and had the program prompt the user to retry the query. The test that attempted to execute an invalid query passed.

Analysis and Discussion

Because we included so many safety checks, the query could fail at several stages, and in most cases, we chose to have the user reprompt the LLM. (We did reprompt the model in one case—if the SQL validator returned a response other than ‘valid’ or ‘invalid’.)} In every other case of failure, we had the user retry their request.) This may not have been the most user-friendly approach, since it involves more troubleshooting on the side of the user (and certain failures may be able to be resolved at the system level (for example, a malformed SQL query could be re-sent to the model with instructions to fix the syntax)). However, we decided that, in the interest of safety, it would be preferable to have the user revise or retry their query rather than reprompt a model with valid or potentially dangerous inputs.

Conclusion and Future Work

Next week, we will further build on this lab by implementing an agent to complete the SQL output from our `query_ollama` model. The agent will execute the SQL queries, advancing the work we did in this lab and making it easier for someone to get what they want from the database.

We did not include a strategy for handling semantic drift because we did not encounter any cases during testing. We felt that having another model reflect on how well-formed and relevant the query was relative to the user's request would not be fruitful, since both models would have likely had the same opinion on the validity of the response. Any misunderstanding of the prompt would likely be due to user error (unclear prompt). We decided that having the model summarize the results would be enough to cue the user into whether the model correctly answered their question, and that the user would retry the correct query on their own, rather than have our program reprompt or restart. However, in the future, we would like to devise a strategy to handle this.

Acknowledgements

We discussed basic plans for our model with our superteam (Alexis, Gabriel, and Dr. B).

References

.